Part I - Short-Answer Questions

Question 1 - Convolution, Padding, and Stride

Input 32 x 32 x 3, 16 filters of size 5 x 5, stride S = 1, padding P = 0.

a. N = 32 (input height/width), F = 5 (filter size), S = 1, P = 0.

b. N_{out} = (N - F + 2P)/S + 1 = (32 - 5 + 0)/1 + 1 = **28**. The feature map is 28 x 28.

c. Each of the 16 filters produces one 28 x 28 map, so the output volume is **28 x 28 x 16**.

d. To keep 32 x 32 with S = 1: (32 - 5 + 2P)/1 + 1 = 32, so 2P = 4 and **P = 2** ("same" padding, P = (F - 1)/2).

Question 2 - CNN Architectures

a. AlexNet: It showed that a deep CNN trained on GPUs could win ImageNet (2012) by a large margin. It popularized ReLU activations, dropout, data augmentation, and GPU training.

b. VGGNet: Stacked 3 x 3 filters give the same receptive field as one large filter (two give 5 x 5, three give 7 x 7) with fewer parameters ($27C^2$ vs. $49C^2$ for one 7 x 7 layer) and more non-linearities (extra ReLUs), making the network deeper and more discriminative.

c. Inception module (GoogLeNet): It applies 1 x 1, 3 x 3, and 5 x 5 convolutions and pooling in parallel and concatenates the outputs, capturing features at multiple scales. 1 x 1 convolutions reduce channels beforehand to keep computation low.

d. ResNet: Residual (skip) connections learn F(x) and output F(x) + x. This addresses the degradation/vanishing-gradient problem in very deep networks: gradients flow through the shortcuts and layers can easily learn an identity mapping, so deeper networks train better instead of worse.

Question 3 - Representation Learning and Transfer Learning

a. Representation learning: Methods where a model automatically learns useful features from raw data instead of relying on hand-engineered features. CNNs learn them end-to-end.

b. Lower vs. higher layers: Lower layers have small receptive fields and see raw pixels, so they learn generic patterns such as edges, colors, and textures. Higher layers combine earlier features over larger receptive fields, learning complex, task-specific features (parts, shapes, whole objects).

c. Transfer learning / fine-tuning: Transfer learning reuses a model trained on a large source task (e.g., ImageNet) as the starting point for a new task. Fine-tuning further trains the pretrained weights (all or some layers) on the new data, typically with a small learning rate.

d. Small dataset: A deep CNN has millions of parameters, so training from scratch on little data overfits and needs heavy compute. A pretrained CNN already holds general features learned from millions of images, so it needs less data, trains faster, and generalizes better.

e. Freezing vs. fine-tuning: A frozen layer has fixed weights (requires_grad = False), so no gradients are computed and no updates happen. A fine-tuned layer remains trainable and is updated by backpropagation. Freezing is cheaper and safer with tiny datasets; fine-tuning is more flexible and often more accurate, but slower and more prone to overfitting.